Modul Git Version Control

Daftar Isi

Apa Itu Version Control? (hal. 1)
Mengenal Git (hal. 2)
Membuat Akun GitHub (hal. 3)
GitHub Student Developer Pack (hal. 4)
Instalasi Git (hal. 5)
Konfigurasi Awal Git (hal. 6)
Konsep Dasar Git (hal. 8)
Perintah Git Dasar (hal. 9)
Bekerja dengan Remote Repository (hal. 11)
Melihat Riwayat dan Membatalkan Perubahan (hal. 13)
Deployment dengan GitHub Pages (hal. 16)
Tips, Trik, dan Best Practice (hal. 26)
Referensi dan Sumber Belajar Lanjutan (hal. 29)

1. Apa Itu Version Control?

Pengertian

Version Control System adalah sistem yang merekam setiap perubahan pada file atau sekumpulan file dari waktu ke waktu, sehingga kamu bisa kembali ke versi tertentu kapan pun kamu mau.

Bayangkan kamu menulis skripsi dan menyimpan file seperti ini:

```
skripsi_v1.docx
skripsi_v2_final.docx
skripsi_v2_final_REVISI.docx
skripsi_v2_final_REVISI_beneran.docx
```

Inilah masalah yang dipecahkan oleh version control — kamu cukup punya **satu file**, dan sistemnya yang melacak semua perubahan.

Kegunaan Version Control

Kegunaan	Penjelasan
Pelacakan Perubahan	Tahu siapa mengubah apa, kapan, dan kenapa
Pemulihan	Kembalikan file ke versi sebelumnya jika ada kesalahan
Kolaborasi	Banyak orang bisa bekerja pada proyek yang sama secara bersamaan
Branching	Uji coba fitur baru tanpa merusak kode utama
Audit Trail	Riwayat lengkap seluruh perubahan proyek

---

2. Mengenal Git

Apa Itu Git?

Git adalah sistem version control terdistribusi (*Distributed Version Control System*) yang paling populer di dunia. Git dibuat oleh **Linus Torvalds** pada tahun 2005 untuk mengelola pengembangan kernel Linux.

Mengapa Git?

- **Cepat** — Sebagian besar operasi berjalan secara lokal
- **Aman** — Setiap data di-hash menggunakan SHA-1
- **Branching yang mudah** — Membuat dan menggabungkan branch sangat ringan
- **Terdistribusi** — Setiap developer punya salinan penuh repository
- **Gratis dan Open Source**

Git vs GitHub

> **Git ≠ GitHub!** Ini perbedaan yang sering membingungkan pemula.

	Git	GitHub
Apa	Software version control	Platform hosting repository
Di mana	Berjalan di komputermu	Layanan cloud (website)
Fungsi	Melacak perubahan kode	Menyimpan & berkolaborasi online
Alternatif	Mercurial, SVN	GitLab, Bitbucket

Analoginya: **Git** = Microsoft Word, **GitHub** = Google Drive.

3. Membuat Akun GitHub

Langkah-Langkah

1. Buka **github.com** di browser
2. Klik **"Sign up"** di pojok kanan atas
3. Pilih metode pendaftaran yang kamu inginkan:
4. **Metode Email (manual)**: masukkan **email** kamu, buat **password** yang kuat, lalu pilih **username** (misal: `namaKamu-dev`)
5. **Metode SSO**: klik **"Continue with Google"** atau **"Continue with Apple"**, lalu ikuti proses login dari akun Google/Apple kamu
6. Selesaikan verifikasi dan klik **"Create account"** (atau lanjutkan flow akun otomatis jika kamu memakai Google/Apple)
7. Pilih plan **Free** (sudah lebih dari cukup untuk pemula)
8. Lengkapi verifikasi email jika diminta

> **Catatan:** Jika kamu mendaftar dengan Google/Apple, GitHub biasanya tetap akan meminta melengkapi **username** dan beberapa pengaturan akun sebelum akun siap dipakai.

Tips Memilih Username GitHub

- Gunakan nama asli atau nama profesional
- Hindari angka atau karakter acak
- Konsisten dengan platform lain (LinkedIn, dll)
- Contoh bagus: `budi-santoso`, `budidev`, `budisantoso`

Personalisasi Profil

Setelah akun dibuat, lengkapi profilmu:

- **Avatar** — Foto profesional atau avatar yang konsisten
- **Bio** — Deskripsi singkat (misal: "Statistics Student | Data Science Enthusiast")
- **Location** — Kota/negara
- **Website** — Link portfolio jika ada
- **README Profile** — File khusus yang tampil di halaman profil GitHub-mu

4. GitHub Student Developer Pack

Apa Itu GitHub Student Developer Pack?

GitHub Student Developer Pack adalah program dari GitHub yang memberikan **akses gratis** ke berbagai tool dan layanan berbayar khusus untuk pelajar/mahasiswa aktif. Nilainya bisa mencapai ribuan dolar!

Cara Mendaftar

1. Buka **education.github.com/pack**
2. Klik **"Get student benefits"**
3. Login dengan akun GitHub kamu
4. Pilih **"Student"**
5. Masukkan **email institusi** kamu (misal: `nama@student.uns.ac.id`)
6. Upload **bukti status mahasiswa** (KTM, surat keterangan, atau transkrip)
7. Tunggu verifikasi (biasanya 1–7 hari kerja)

> **Tips:** Gunakan email kampus jika ada. Jika tidak, upload foto KTM yang jelas.

Benefit Unggulan

Tool	Benefit	Nilai
GitHub Pro	Repository private tak terbatas, fitur advanced	Gratis selama kuliah
GitHub Copilot	AI coding assistant	Gratis selama kuliah
JetBrains IDEs	PyCharm, IntelliJ, dll	Gratis 1 tahun (renewable)
Namecheap	Domain `.me` gratis 1 tahun + SSL	~$13/tahun
Microsoft Azure	$100 Azure credits	$100
DigitalOcean	Cloud server credits	$200
Canva Pro	Design tool premium	Gratis
DataCamp	Kursus data science	Gratis 3 bulan
MongoDB Atlas	Cloud database	Credits gratis

---

5. Instalasi Git

Windows

Cara 1: Download Installer (Direkomendasikan)

1. Buka **git-scm.com/download/win**
2. Download installer terbaru (64-bit)
3. Jalankan installer dan ikuti wizard:
   - Biarkan semua pengaturan default
   - Pada bagian **"Adjusting your PATH environment"**, pilih **"Git from the command line and also from 3rd-party software"**
   - Pada bagian **"Choosing the default behavior of `git pull`"**, pilih **"Fast-forward or merge"**
4. Klik **Install** lalu **Finish**

Cara 2: Via winget (Windows Package Manager)

CMD / TERMINAL

```
winget install --id Git.Git -e --source winget
```

macOS

Cara 1: Homebrew (Direkomendasikan)

Jika Homebrew belum terpasang, instal dulu:

BASH / TERMINAL

```
/bin/bash -c "$(
  curl -fsSL \
    https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh
)"
```

Setelah itu, instal Git:

BASH / TERMINAL

```
brew install git
```

Cara 2: Installer Resmi

1. Buka **git-scm.com/download/mac**
2. Download installer yang sesuai dengan versi macOS-mu
3. Jalankan installer lalu ikuti langkah yang muncul

> Jika Git bawaan macOS terlalu lama versinya, Homebrew biasanya memberi versi yang lebih baru dan lebih mudah diperbarui.

Verifikasi Instalasi

Buka **Command Prompt (CMD)** di Windows atau **Terminal** di macOS, lalu jalankan:

CMD / TERMINAL

```
git --version
```

Output yang diharapkan:

```
git version 2.47.0.windows.1
```

---

6. Konfigurasi Awal Git

Sebelum mulai bekerja, kita perlu mengatur identitas di Git. Informasi ini akan muncul di setiap commit yang kamu buat.

Mengatur Nama dan Email

CMD / TERMINAL

```
git config --global user.name "Nama Kamu"
git config --global user.email "email@kamu.com"
```

> Gunakan email yang sama dengan akun GitHub-mu.

Mengatur Default Branch Name

CMD / TERMINAL

```
git config --global init.defaultBranch main
```

Mengatur Editor Teks Default (Opsional)

CMD / TERMINAL

```
:: Menggunakan Notepad (bawaan Windows)
git config --global core.editor notepad

:: Menggunakan VS Code
git config --global core.editor "code --wait"
```

Mengatur Line Ending (Penting untuk Windows!)

CMD / TERMINAL

```
git config --global core.autocrlf true
```

> Ini mencegah masalah perbedaan format baris antara Windows dan Linux/Mac.

Jika kamu memakai macOS atau Linux, gunakan konfigurasi ini:

BASH / TERMINAL

```
git config --global core.autocrlf input
```

Melihat Semua Konfigurasi

CMD / TERMINAL

```
git config --list
```

File Konfigurasi

Konfigurasi global tersimpan di `C:\Users\NamaUser\.gitconfig`. Kamu bisa membukanya dengan:

CMD / TERMINAL

```
notepad C:\Users\%USERNAME%\.gitconfig
```

---

7. Konsep Dasar Git

Tiga Area Kerja Git

```
┌──────────────────────────────────────────────────────────┐
│                     WORKING DIRECTORY                    │
│          (File yang sedang kamu edit sekarang)           │
└──────────────────────────┬───────────────────────────────┘
                           │  git add
                           ▼
┌──────────────────────────────────────────────────────────┐
│                     STAGING AREA (INDEX)                 │
│        (File yang sudah siap untuk di-commit)            │
└──────────────────────────┬───────────────────────────────┘
                           │  git commit
                           ▼
┌──────────────────────────────────────────────────────────┐
│                     REPOSITORY (.git)                    │
│          (Riwayat commit yang sudah tersimpan)           │
└──────────────────────────────────────────────────────────┘
```

Area	Penjelasan
Working Directory	Folder proyekmu — tempat kamu membuat dan mengedit file
Staging Area	"Antrian commit" — file yang sudah dipilih untuk disimpan
Repository	Database Git — semua riwayat perubahan tersimpan di sini

Status File dalam Git

```
Untracked → Tracked (Staged) → Committed → Modified → Staged → Committed
```

- **Untracked** — File baru yang belum dikenal Git
- **Staged** — File sudah ditambahkan ke staging area
- **Committed** — File sudah tersimpan di repository
- **Modified** — File yang sudah di-commit tapi ada perubahan baru

Apa Itu Commit?

Commit adalah snapshot (foto) dari seluruh proyekmu pada suatu titik waktu. Setiap commit memiliki:

- **Hash unik** (misal: `a3f2c1d`) — pengenal commit
- **Pesan** — deskripsi apa yang berubah
- **Timestamp** — kapan commit dibuat

- **Author** — siapa yang membuat commit
- **Parent** — pointer ke commit sebelumnya

---

8. Perintah Git Dasar

Membuat Repository Baru

CMD / TERMINAL

```
:: Buat folder proyek baru
mkdir nama-proyek
cd nama-proyek

:: Inisialisasi Git repository
git init
```

Output:

```
Initialized empty Git repository in C:/Users/kamu/nama-proyek/.git/
```

Melihat Status Repository

CMD / TERMINAL

```
git status
```

Menambahkan File ke Staging Area

CMD / TERMINAL

```
:: Tambahkan file tertentu
git add nama-file.txt

:: Tambahkan semua file di folder saat ini
git add .

:: Tambahkan semua file dengan ekstensi tertentu
git add *.py
```

Membuat Commit

CMD / TERMINAL

```
:: Commit dengan pesan singkat
git commit -m "Pesan commit yang deskriptif"

:: Contoh pesan commit yang baik
git commit -m "Add login feature with email validation"
git commit -m "Fix bug: null pointer exception on user profile"
git commit -m "Update README with installation instructions"
```

> **Tips Pesan Commit:** Gunakan present tense dalam bahasa Inggris. Jelaskan **apa** yang berubah, bukan **bagaimana**.

Shortcut: Add + Commit Sekaligus

CMD / TERMINAL

```
:: Hanya untuk file yang sudah pernah di-track sebelumnya
git commit -am "Pesan commit"
```

Melihat Log Commit

CMD / TERMINAL

```
:: Log lengkap
git log

:: Log ringkas (satu baris per commit)
git log --oneline

:: Log dengan grafik branch
git log --oneline --graph --all

:: Log N commit terakhir
git log --oneline -5
```

Contoh Alur Kerja Lengkap

CMD / TERMINAL

```
:: 1. Masuk ke folder proyek
cd C:\Users\kamu\proyek-saya

:: 2. Inisialisasi Git
git init

:: 3. Buat file baru (contoh)
echo. > README.md
notepad README.md

:: 4. Cek status
git status

:: 5. Tambahkan ke staging
git add README.md

:: 6. Commit
git commit -m "Initial commit: add README"

:: 7. Cek log
git log --oneline
```

---

9. Bekerja dengan Remote Repository

Apa Itu Remote?

Remote adalah versi repository yang tersimpan di server (seperti GitHub). Ini memungkinkan kolaborasi dan backup kode.

Membuat Repository di GitHub

1. Login ke **github.com**
2. Klik tombol **"+"** → **"New repository"**
3. Isi **Repository name** (gunakan huruf kecil, pisahkan dengan `-`)
4. Pilih **Public** atau **Private**
5. **Jangan** centang "Initialize this repository" jika sudah punya repo lokal
6. Klik **"Create repository"**

Menghubungkan Repo Lokal ke GitHub

CMD / TERMINAL

```
:: Tambahkan remote bernama "origin"
git remote add origin https://github.com/username/nama-repo.git

:: Verifikasi remote
git remote -v

:: Push (kirim) ke GitHub untuk pertama kali
git push -u origin main
```

Autentikasi GitHub

GitHub tidak lagi menerima password biasa. Gunakan salah satu cara berikut:

Cara 1: Personal Access Token (PAT)

1. Buka **GitHub** → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)**
2. Klik **"Generate new token"**
3. Centang scope **"repo"** (minimal)
4. Copy token yang dihasilkan
5. Gunakan token ini sebagai pengganti password saat diminta

Cara 2: GitHub CLI (Direkomendasikan)

CMD / TERMINAL

```
:: Install GitHub CLI via winget
winget install --id GitHub.cli

:: Login
gh auth login
```

Ikuti instruksi di layar, pilih **GitHub.com** → **HTTPS** → **Login with a web browser**.

Perintah Remote Penting

CMD / TERMINAL

```
:: Clone repository dari GitHub ke lokal
git clone https://github.com/username/nama-repo.git

:: Clone ke folder dengan nama berbeda
git clone https://github.com/username/nama-repo.git nama-folder-baru

:: Pull (ambil perubahan terbaru dari remote)
git pull origin main

:: Push (kirim perubahan lokal ke remote)
git push origin main

:: Lihat semua remote
git remote -v

:: Hapus remote
git remote remove origin
```

Alur Kerja Sehari-Hari

CMD / TERMINAL

```
:: Setiap mulai kerja — ambil perubahan terbaru
git pull origin main

:: ... kerjakan tugasmu ...

:: Setelah selesai — kirim ke GitHub
git add .
git commit -m "Deskripsi perubahan"
git push origin main
```

---

10. Melihat Riwayat dan Membatalkan Perubahan

Melihat Riwayat Perubahan

Saat bekerja dengan Git, kamu perlu tahu cara melihat apa yang berubah, siapa yang mengubah, dan kapan perubahan itu dibuat. Perintah-perintah berikut paling sering dipakai:

CMD / TERMINAL

```
:: Lihat riwayat commit singkat
git log --oneline

:: Lihat riwayat commit dalam bentuk grafik
git log --oneline --graph --all

:: Lihat perubahan yang belum di-staging
git diff

:: Lihat perubahan yang sudah di-staging
git diff --staged

:: Lihat detail satu commit tertentu
git show abc1234
```

Gunakan `git diff` saat ingin mengecek isi perubahan sebelum commit, dan `git log` saat ingin menelusuri riwayat repository.

Membatalkan Perubahan dengan Aman

Cara membatalkan perubahan tergantung pada posisi perubahan tersebut: masih di working directory, sudah di-staging, atau sudah terlanjur menjadi commit.

CMD / TERMINAL

```
:: SEBELUM STAGING

:: Batalkan perubahan pada file dan kembalikan ke versi commit terakhir
git restore nama-file.txt


:: SETELAH STAGING, SEBELUM COMMIT

:: Keluarkan file dari staging, tetapi perubahan file tetap dipertahankan
git restore --staged nama-file.txt


:: SETELAH COMMIT

:: Buat commit baru yang membatalkan commit tertentu
git revert abc1234

:: Hapus commit terakhir, tetapi perubahan tetap berada di staging area
git reset --soft HEAD~1

:: Hapus commit terakhir, perubahan kembali ke working directory
git reset --mixed HEAD~1

:: Hapus commit terakhir beserta semua perubahannya
git reset --hard HEAD~1
```

> **Peringatan:** `git reset --hard` menghapus perubahan secara permanen. Hindari memakai perintah ini pada repository bersama atau pada commit yang sudah di-push, kecuali kamu benar-benar paham risikonya.

Memperbaiki Commit Terakhir

Jika kamu baru saja commit lalu sadar ada pesan yang salah atau ada file yang belum ikut, kamu bisa memperbaikinya tanpa membuat commit baru.

CMD / TERMINAL

```
:: Ubah pesan commit terakhir
git commit --amend -m "Pesan yang sudah diperbaiki"

:: Tambahkan file yang terlupa ke commit terakhir
git add file-yang-terlupa.txt
git commit --amend --no-edit
```

> **Catatan:** `git commit --amend` aman untuk commit lokal yang belum di-push. Jika commit tersebut sudah di-push ke branch bersama, gunakan dengan hati-hati.

---

11. Deployment dengan GitHub Pages

Mengapa Deployment Penting?

Belajar Git akan terasa lebih utuh kalau hasil kerja tidak berhenti di laptop sendiri. Dengan deployment, repository yang kamu kelola bisa berubah menjadi website, dokumentasi, atau laporan analisis yang dapat diakses dosen, teman satu tim, dan publik.

Untuk kebutuhan perkuliahan dan proyek data, **GitHub Pages** adalah pilihan yang sangat praktis karena:

- gratis
- terhubung langsung dengan repository GitHub
- cocok untuk website statis, dokumentasi, dan laporan analisis
- cukup mudah untuk pemula, tetapi tetap relevan untuk workflow yang lebih profesional

Apa Itu GitHub Pages?

GitHub Pages adalah layanan hosting gratis dari GitHub untuk menayangkan website statis langsung dari repository.

Contoh URL yang dihasilkan:

```
https://username.github.io/nama-repo/
```

Jenis GitHub Pages yang paling umum:

Tipe	Repository	URL
User/Org site	`username.github.io`	`https://username.github.io`
Project site	Repository biasa	`https://username.github.io/nama-repo`

Kapan Menggunakan Pendekatan yang Mana?

Kebutuhan	Pendekatan	Cocok Untuk
Website HTML/CSS/JS sederhana	Publish dari branch `main` ke `/root` atau `/docs`	Pemula
Laporan `R Markdown` satu halaman	Render lokal ke folder `docs/`, lalu push	Tugas/laporan cepat
Website analisis multi-halaman	Render ke `docs/` dengan `render_site()` atau Quarto	Proyek yang lebih rapi
Deploy otomatis setiap ada push	GitHub Actions + GitHub Pages	Workflow yang lebih profesional

Persiapan Sebelum Deploy

Sebelum menekan tombol publish, pastikan:

1. Repository sudah ada di GitHub.
2. Halaman utama akan menjadi `index.html`.
3. Semua gambar, CSS, dan JavaScript menggunakan **path relatif**, bukan path lokal komputer.
4. Hasil render sudah dicek dulu di browser lokal.
5. Kamu konsisten memilih salah satu sumber deploy: `docs/` di branch tertentu atau workflow GitHub Actions.

11.1 Deploy Website HTML Sederhana

Ini pendekatan paling mudah untuk website statis biasa.

Contoh struktur proyek:

```
nama-repo/
|-- index.html
|-- style.css
|-- script.js
`-- assets/
    `-- gambar.png
```

Langkah-langkah:

1. Buat atau buka repository di GitHub.
2. Pastikan ada file `index.html` di root proyek atau di folder `docs/`.
3. Push semua file ke branch utama, biasanya `main`.
4. Buka **Settings** repository -> **Pages**.
5. Pada bagian **Build and deployment**, pilih **Deploy from a branch**.
6. Pilih branch `main`, lalu pilih folder `/(root)` atau `/docs`.
7. Klik **Save** dan tunggu beberapa menit.

> **Catatan:** GitHub Pages akan mencari `index.html` sebagai halaman awal. Jika nama file utamamu bukan `index.html`, website bisa tampil 404 atau hanya bisa diakses lewat URL file lengkap.

11.2 Deploy R Markdown Secara Manual

Pendekatan ini sangat cocok untuk tugas analisis data atau laporan yang hanya perlu diperbarui sesekali.

Alur kerjanya:

```
analisis.Rmd -> render di lokal -> hasil HTML masuk ke docs/ -> push ke
GitHub -> GitHub Pages menayangkan docs/
```

Contoh struktur proyek:

```
proyek-analisis/
|-- analisis.Rmd
|-- data/
|   `-- dataset.csv
`-- docs/
```

Contoh YAML header pada file `.Rmd`:

YAML

```yaml
---
title: "Judul Analisis"
author: "Namamu"
date: "`r Sys.Date()`"
output:
  html_document:
    toc: true
    toc_float: true
    code_folding: show
    self_contained: true
---
```

Mengapa `self_contained: true` penting?

- Semua aset pendukung akan tertanam dalam satu file HTML.
- Risiko gambar atau CSS hilang saat dipublish jadi lebih kecil.
- Sangat cocok untuk laporan satu halaman.

Render file ke folder `docs/`:

R

```r
rmarkdown::render(
  input = "analisis.Rmd",
  output_file = "index.html",
  output_dir = "docs"
)
```

Push hasil render ke GitHub:

CMD / TERMINAL

```
git add analisis.Rmd docs/
git commit -m "docs: publish laporan analisis ke GitHub Pages"
git push origin main
```

Aktifkan GitHub Pages:

1. Buka **Settings** -> **Pages**.
2. Pada **Build and deployment**, pilih **Deploy from a branch**.
3. Pilih branch `main` dan folder `/docs`.
4. Klik **Save**.

Jika semua benar, website akan tayang di URL project site repository-mu.

11.3 Deploy Otomatis dengan GitHub Actions

Jika kamu tidak ingin render manual setiap kali ada perubahan, gunakan **GitHub Actions**. Workflow ini akan:

1. mengambil isi repository,
2. menjalankan proses build atau render,
3. mengunggah hasil build,
4. lalu mendeploy hasilnya ke GitHub Pages.

Ini lebih rapi untuk proyek yang sering diperbarui atau dikerjakan berulang.

Contoh workflow untuk `R Markdown`:

Buat file `.github/workflows/deploy-pages.yml`

YAML

```yaml
name: Build and Deploy GitHub Pages

on:
  push:
    branches: [ main ]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Setup R
        uses: r-lib/actions/setup-r@v2

      - name: Setup Pandoc
        uses: r-lib/actions/setup-pandoc@v2

      - name: Install R packages
        uses: r-lib/actions/setup-r-dependencies@v2
        with:
          packages: |
            any::rmarkdown
            any::knitr

      - name: Render R Markdown
        run: |
          mkdir -p docs
          Rscript -e "rmarkdown::render('analisis.Rmd',
output_file='index.html', output_dir='docs')"
```

```
      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v4
        with:
          path: ./docs

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

Pengaturan penting setelah workflow dibuat:

1. Buka **Settings** -> **Pages**.
2. Pada **Source**, pilih **GitHub Actions**.
3. Push perubahan ke branch `main`.
4. Pantau tab **Actions** sampai workflow selesai.

Kapan pendekatan otomatis lebih baik?

- Saat laporan sering diperbarui.
- Saat proyek dikerjakan lebih dari satu orang.
- Saat kamu ingin proses publish konsisten dan minim langkah manual.

11.4 Website Multi-Halaman dengan R Markdown atau Quarto

Kalau proyekmu lebih dari satu halaman, jangan paksa semuanya ke satu file besar. Lebih baik buat website kecil yang terstruktur.

Opsi A: `rmarkdown::render_site()`

Struktur proyek:

```
proyek-website/
|-- _site.yml
|-- index.Rmd
|-- eda.Rmd
|-- modeling.Rmd
`-- kesimpulan.Rmd
```

Contoh file `_site.yml`:

YAML

```yaml
name: "Analisis Data"
output_dir: docs
navbar:
  title: "Proyek Analisis"
  left:
    - text: "Beranda"
      href: index.html
    - text: "EDA"
      href: eda.html
    - text: "Modeling"
      href: modeling.html
    - text: "Kesimpulan"
      href: kesimpulan.html

output:
  html_document:
    toc: true
    toc_float: true
    self_contained: false
```

Render seluruh situs:

R

```r
rmarkdown::render_site()
```

> **Catatan:** Jika `self_contained: false`, pastikan seluruh file pendukung hasil render ikut terpush ke `docs/`.

Opsi B: Quarto

Quarto adalah pilihan modern untuk laporan dan website data. Ia cocok untuk R, Python, dan kombinasi keduanya.

Contoh file `_quarto.yml`:

YAML

```yaml
project:
  type: website
  output-dir: docs

website:
  title: "Analisis Klaim Asuransi"
  navbar:
    left:
      - href: index.qmd
        text: Beranda
      - href: eda.qmd
        text: EDA
      - href: modeling.qmd
        text: Modeling

format:
  html:
    toc: true
    code-fold: true
```

Render website Quarto:

CMD / TERMINAL

```
quarto render
```

Setelah itu, kamu bisa:

- publish dari folder `docs/`, atau
- mengganti langkah render di workflow GitHub Actions dengan `quarto render`

11.5 Checklist Sebelum Publish

Gunakan checklist ini sebelum deploy:

```
[ ] File utama bernama index.html
[ ] Semua aset memakai path relatif
[ ] Folder output sudah benar (docs/ atau artifact build)
[ ] Tidak ada data sensitif di repository
[ ] Hasil website/laporan sudah dicek di browser lokal
[ ] Pengaturan Pages sudah sesuai dengan metode deploy yang dipilih
```

11.6 Troubleshooting Umum

Masalah	Penyebab yang Sering Terjadi	Solusi
Halaman 404	Tidak ada `index.html` di lokasi publish	Pastikan file output bernama `index.html`
Gambar/CSS tidak muncul	Path masih absolut atau file aset tidak ikut ter-push	Gunakan path relatif dan cek folder output
Situs tidak berubah	Belum selesai build, cache browser, atau source salah	Tunggu beberapa menit, refresh keras, cek Settings -> Pages
Workflow gagal	Package/dependency belum lengkap	Tambahkan dependency yang diperlukan di workflow
File `docs/` tidak terbaca	Folder output salah atau belum dibuat	Pastikan render menghasilkan file ke `docs/`

11.7 Alur Praktis yang Disarankan

Untuk pemula, urutan kerja yang paling aman biasanya seperti ini:

1. Kerjakan laporan atau website di lokal.
2. Render hasil ke `docs/`.
3. Cek hasilnya di browser lokal.
4. Commit dan push ke GitHub.
5. Aktifkan GitHub Pages dari `/docs`, atau gunakan GitHub Actions jika ingin otomatis.

Dengan alur ini, Git berfungsi sebagai pencatat versi, sedangkan GitHub Pages menjadi media publikasinya.

---

12. Tips, Trik, dan Best Practice

File `.gitignore`

File `.gitignore` memberitahu Git untuk **mengabaikan** file atau folder tertentu agar tidak ikut dilacak.

CMD / TERMINAL

```
:: Buat file .gitignore
notepad .gitignore
```

Contoh `.gitignore` untuk proyek Python atau analisis data:

.GITIGNORE

```
# Python
__pycache__/
*.py[cod]
*.pyo
.env
venv/
env/

# IDE
.vscode/
.idea/

# Sistem operasi
.DS_Store
Thumbs.db

# Build dan log
*.log
dist/
build/

# R / RStudio
.Rhistory
.RData
.Rproj.user/
*_cache/
*_files/

# Rahasia / credential
secrets.json
.env.local
```

> **Aturan emas:** Jangan pernah commit password, API key, token, atau file credential ke Git.

Git Aliases

Alias membantu mempercepat perintah yang sering dipakai:

CMD / TERMINAL

```
git config --global alias.st status
git config --global alias.cm "commit -m"
git config --global alias.lg "log --oneline --graph --all --decorate"

:: Contoh penggunaan
git st
git cm "docs: rapikan struktur modul"
git lg
```

Conventional Commits

Format commit yang rapi akan sangat membantu saat membaca riwayat perubahan.

```
<type>: <description>
```

Atau jika ingin lebih spesifik:

```
<type>(<scope>): <description>
```

Tipe commit yang paling umum:

Type	Kapan Digunakan
`feat`	Menambah fitur atau konten baru
`fix`	Memperbaiki bug atau kesalahan
`docs`	Perubahan dokumentasi
`refactor`	Merapikan struktur tanpa mengubah hasil
`test`	Menambah atau memperbaiki pengujian
`chore`	Tugas pemeliharaan kecil

Contoh:

```
docs: tambah panduan deployment GitHub Pages
fix: perbaiki path output laporan
feat(report): tambah halaman kesimpulan
```

Perintah Ringkas yang Sering Dipakai

CMD / TERMINAL

```
git status
git add .
git commit -m "pesan"
git log --oneline
git diff
git remote -v
git push origin main
```

Checklist Sebelum Push

```
[ ] File sensitif sudah aman
[ ] .gitignore sudah benar
[ ] Pesan commit jelas dan deskriptif
[ ] Hasil akhir sudah dicek
[ ] File yang di-push memang file yang dibutuhkan
```

---

13. Referensi dan Sumber Belajar Lanjutan

Dokumentasi Resmi

- **Git Documentation** - git-scm.com/doc
- **Pro Git Book** - git-scm.com/book
- **GitHub Docs** - docs.github.com
- **GitHub Pages Docs** - docs.github.com/pages
- **Quarto Docs** - quarto.org/docs

Latihan dan Pembelajaran Interaktif

- **Learn Git Branching** - learngitbranching.js.org
- **GitHub Skills** - skills.github.com
- **Oh My Git!** - game untuk belajar Git secara visual

Cheat Sheet Singkat

CMD / TERMINAL

```
:: SETUP
git config --global user.name "Nama"
git config --global user.email "email@mail.com"

:: MEMULAI REPOSITORY
git init
git clone <url>

:: PERUBAHAN HARIAN
git status
git add <file>
git add .
git commit -m "pesan"
git log --oneline

:: REMOTE
git remote add origin <url>
git push -u origin main
git pull origin main

:: DEPLOYMENT
quarto render
git add docs/
git commit -m "docs: update website"
git push origin main
```

Catatan Akhir

Git adalah skill yang paling cepat dikuasai lewat praktik langsung. Mulailah dari workflow sederhana: edit file, commit, push, lalu publish hasilnya. Setelah itu, barulah perlahan naik ke workflow yang lebih otomatis.

"*Commit early, commit often.*" - Git mantra

Modul ini dibuat sebagai panduan belajar Git dari nol hingga mahir. Versi terakhir diperbarui: April 2026.